Java Referenz

Autor: Markus Schalk

Kompiliert / Interpretiert

- Java Quellcode kompiliert in Java Bytecode
- Java Quellcode wird nicht wie in Python interpretiert

Typisierung

Java ist statisch und stark typisiert. Variablen müssen bei der Deklaration einen Typ angeben.

Hingegen ist Python dynamisch typisiert. Variablen werden zur Laufzeit typisiert.

Eigenschaften der OO (in Java)

- Vererbung
- Abstraktion
- Kapselung
- Polymorphismus (Vielgestaltigkeit)

Plus:

- Wiederverwendbarkeit (DRY – don't repeat yourself)
- Trennung der Verantwortlichkeiten (separation of concerns)

Abstraktion in Java

Klassen sind die allgemeine Beschreibung für ein Objekt, ein *Template*. Sie enthalten die Beschreibung der Eigenschaften (Variablen), das Verhalten (Methoden), und die Relationen zu anderen Klassen.

Ein Objekt wird als Instanz aus einer Klasse erzeugt.

Aufgrund der statischen Typisierung muss in Java der Datentyp explizit angegeben werden.

```java
public class Player {
    // information hiding
    // no direct access to (instance) variables
    // getter / setter public
    private String name;
    private int score = 0;

    public void setName(String name) {
        this.name = name;
    }
    public String getName() {
        return name;
    }

    // constructor
    public Player(String name) {
        this.name = name;
    }

    public void updateScore(int points) {
        score += points;
    }
}
```

```java
public static void main(String[] args) {
    Player p = new Player("Tom");
    p.setName("Thomas");
    p.updateScore(1);
}
```

Vererbung in Java

```java
class User {
    protected String name;
}
class AdminUser extends User {
    public void rootLogin() {}
}
```

Interfaces

Interfaces erlauben, dass bestimmte Fähigkeiten bzw. Eigenschaften in Klassen, welche das Interface implementieren, enthalten sind. Ein Interface enthält keine Implementierung. Aus diesem Grund ist ein Interface immer abstract, was nicht explizit deklariert werden muss. Daher sind alle Methodendeklarationen ebenfalls abstract.

Interfaces erlauben Mehrfachvererbung, da es Java nicht erlaubt, eine Klasse von mehreren Superklassen abzuleiten.

```
interface Foo {
    int a();
}
interface Bar extends Foo {
    int b();
}

class AClass implements Bar {
    int a() {}
    int b() {}
}
```

Abstrakte Klassen

Abstrakte Klassen können nicht instanziiert werden, aber man kann Subklassen davon definieren. Abstrakte Klassen können abstrakte Methoden enthalten, müssen aber nicht. Alle abstrakten Methoden müssen in Subklassen implementiert werden. Ist dies jedoch nicht der Fall, muss die Unterklasse ebenfalls als abstrakt deklariert werden.

```
abstract class GraphicObject {
    int x;
    int y;

    void moveTo(int newX, int newY) {}

    abstract void draw();
    abstract void resize();
}

class Circle extends GraphicObject {
    void draw() {}
    void resize() {}
}
class Rectangle extends GraphicObject {
    void draw() {}
    void resize() {}
}
```

Abstrakte Klassen vs. Interfaces

Abstrakte Klassen sind Interfaces ähnlich. Abstrakte Klassen können nicht instanziiert werden und sie können eine Mischung von Methoden enthalten, die mit oder ohne Implementierung deklariert sind. Abstrakte Klassen erlauben jedoch Felder zu deklarieren, die nicht statisch und final sind, und konkrete, public, protected und private Methoden zu definieren. Bei Interfaces sind alle Felder automatisch public, static und final. Alle deklarierten Methoden sind public.

Außerdem kann nur eine Klasse erweitert werden, unabhängig davon, ob sie abstrakt ist oder nicht, während eine beliebige Anzahl von Interfaces implementiert werden kann.

Schlüsselwort `final`

Werden Variablen mit final gekennzeichnet, sind sie nach der Initialisierung nicht mehr änderbar. Sie können jedoch als uninitialisierte, leere Variablen deklariert, müssen dann jedoch im Konstruktor initialisiert werden.

Werden Methoden als final gekennzeichnet, können sie in abgeleiteten Klassen nicht überschrieben werden. Die Anwendungsgründe sind Sicherheit bzw. Effizienzsteigerung (kein dynamisches Binden).

Bei Methodenparametern verhält es sich ähnlich wie bei Variablen. Werden sie final deklariert, sind sie nach der Übergabe nicht mehr veränderbar.

Wird eine Klasse mit final gekennzeichnet, kann von ihr keine Subklasse abgeleitet werden. Die Anwendungsgründe sind Sicherheit bzw. Effizienz soll erhöht werden. Methoden einer final definierten Klasse sind ebenfalls implizit als final definiert.

Overloading vs. Overriding

Overriding liegt vor, wenn die Signatur der Methode in der Super- und in der Subklasse gleich ist. Overloading liegt vor, wenn zwei oder mehr Methoden in derselben Klasse denselben Namen, aber unterschiedliche Parameter haben.

```
public void make() {}
public void make(String s) {}

@Override
public String toString() {
    return "User: " + name;
}
```

Polymorphismus

Zuweisung mit Subklasse / Interface

Wegen der Kompatibilität kann eine Instanz der Subklasse auch einer Superklassenvariablen zugewiesen werden. Das funktioniert ebenso für Interfaces, wobei ein Interface definiert und einer Implementierung zugewiesen wird.

```
User user = new AdminUser();
```

Typenüberprüfung

```
if (user instanceof AdminUser)
```

Typumwandlung (Casting)

```
AdminUser admin = (AdminUser) user;
```

Klasse `Object`

Die Klasse Object ist die Stammklasse aller anderen Klassen und sorgt dafür, dass einige elementare Methoden wie toString(), equals(), hashCode()... in allen Klassen zur Verfügung stehen.

Collections

```
List<AdminUser> list = new ArrayList<AdminUser>();
```

Vergleich von Objekten in Java

Die Methoden equals() und hashCode() sind in Java für den Vergleich von Objekten zuständig.

equals() Methode

Die equals() Methode dient dazu, zwei Objekte auf inhaltliche Gleichheit zu überprüfen. Die Standardimplementierung in der Object-Klasse vergleicht nur die Objektreferenzen im Speicher. Um sinnvolle Objektvergleiche zu ermöglichen, wird die equals() Methode überschrieben.

hashCode() Methode

Die hashCode() Methode generiert einen numerischen Hashwert für ein Objekt. DIe überschriebene Implementierung muss konsistent mit equals() sein: Wenn zwei Objekte laut equals() gleich sind, müssen sie den gleichen Hashcode haben. Wenn equals() überschrieben wird, muss auch hashCode() überschrieben werden

Beispielimplementierung für eine Klasse Person:

```
@Override
public int hashCode() {
    return Objects.hash(name, age);
}
```

Literatur

- Offizielle Oracle Java Dokumentation
- Java ist auch eine Insel von Christian Ullenboom
- Java Beginners
- Vogella
- Maven Build Lifecycle

Projektstruktur

Die Projektstruktur beinhaltet unter `main` den Produktivcode und unter `test` den Testcode. Darunter liegen die Packages (e.g. org.se.lab). Neue Java-Klassen für den Produktivcode können im Package mittels Kontextmenü hinzugefügt werden.

Testfälle können mittels Rechtsklick aufs Package im Ordner des Testcode unter *Run Tests* ausgeführt werden.

Die gegebenen Projekte für die Übungen können über *File/Open* (e.g. UserManagement, ShoppingCart) geöffnet werden. Bei funktionierender Entwicklungsumgebung wird das Projekt erfolgreich konfiguriert und die Testfälle sind ausführbar.

Öffnen Sie die Java Projekte immer einzeln und nicht den übergeordneten Ordner.

Mob Programming

Als Mob Programming bezeichnet man einen kollaborativen Ansatz im Software Engineering. Meist löst ein Team eine Aufgabe gleichzeitig, an einem Ort oder via Bildschirmfreigabe, an einem einzigen Computer. Mob Programming versteht sich als Evolution des Pair Programmings. Nicht nur zwei sondern alle verfügbaren Entwickler kümmern sich um die Lösung einer Aufgabe.

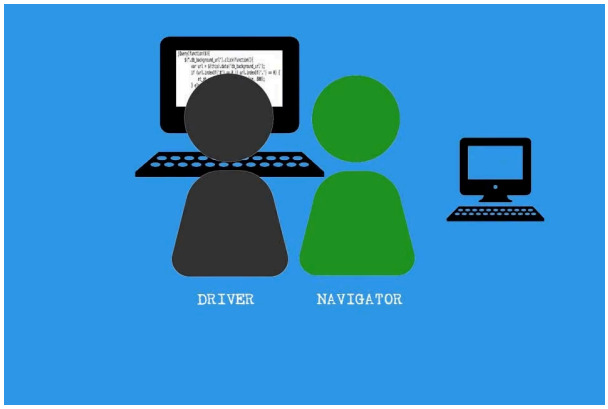
Die **Rollen** beim Mob Programming sind temporär und wechselnd. Folgende zwei Rollen gibt es bei jedem Mob Programming:

- Driver
- Navigator

Der **Driver** sitzt vor dem Rechner, tippt den Quellcode und setzt die Aussagen, Anforderungen, Ideen der anderen Teammitglieder um.

Die **Navigatoren** sind die Teammitglieder, welche Ideen entwickeln, miteinander diskutieren und Vorschläge zu einer Lösung machen. Daher liefern sie Beiträge als Input für den Driver.

`Wichtig` ist, dass die Rollen wechselnd sind, sodass jeder/jede eine Aufgabe als Driver übernimmt.



Quelle: https://t2informatik.de/wissen-kompakt/mob-programming/